



MEF University
Department of Computer Engineering
COMP 451 Introduction to Natural Language Processing Project - 1:
Aspect-Based Sentiment Analysis

Group Members

Begüm Mina Bestelci - 042101104
Buse Dikici - 042101095
Gözde Yıldız - 042101103
Rabia Lafcı - 042101092
Semih Yılmaz - 042101116

1. Project Description

This project investigates and compares annotation methods and prompting techniques for Aspect-Based Sentiment Analysis (ABSA). Initially, an annotated dataset (Dataset1) was sourced online as a benchmark. From this dataset, 100-200 samples were extracted, stripped of annotations, and re-annotated using prompting strategies like zero-shot, few-shot, and chain-of-thought with advanced language models such as ChatGPT, Claude, and Gemini. The re-labeled data was compared with the original dataset to evaluate the performance via accuracy, precision, F1-score, and recall metrics.

In the second stage, a new dataset (Dataset2) consisting of 300 entries was created by scanning the comments of a YouTube video with Selenium and ChromeDriver. This dataset was annotated with zero-shot, few-shot, and chain-of-thought methods using ChatGPT. The constraints of the aspect values were limited to the topics of the video comments, and data were saved in JSON format for consistency.

This report details the stages of developing and evaluating a dataset for annotation purposes. It includes the prompts used for annotation, websites utilized for data collection, a description of how the crawler operates, and an analysis of the success/similarity rate of prompts against Dataset 1. The results offer valuable insights into the strengths and limitations of modern NLP tools in fine-grained sentiment analysis.

2. Final Prompts Used for Annotation

For Dataset 1, a raw text file (first_100_texts.json) was provided in JSON format. This raw text was annotated by the ChatGPT, Gemini and Claude models using the following prompts for the zeroshot, fewshot and chain-of-thought (CoT) approaches respectively.

1. Prompt 1:

Zero shot:

Analyze the following review and identify the aspects discussed. The aspects can belong to the following categories: ambience, food, menu, miscellaneous, place, price, service, and staff. For each aspect mentioned, assign a sentiment (positive, negative, or neutral). Provide the output in JSON format.

Review: "The atmosphere was wonderful, but the service and food were terrible."

2. Prompt 2:

Few Shot:

Analyze the following review for aspects and sentiments. The aspects can belong to the following categories: ambience, food, menu, miscellaneous, place, price, service, and staff. For each aspect, determine its sentiment (positive, negative, or neutral). Refer to the examples for guidance.

Example 1:

Review: "The food was delicious, but the service was slow."

Output: {"aspects": [{"aspect": "food", "sentiment": "positive"}, {"aspect": "service", "sentiment": "negative"}]}

Example 2:

Review: "The prices were fair, and the ambience was relaxing."

Output: {"aspects": [{"aspect": "price", "sentiment": "positive"}, {"aspect": "ambience", "sentiment": "positive"}]}

Example 3:

Review: "The waiter was rude, and the menu options were limited."

Output: {"aspects": [{"aspect": "staff", "sentiment": "negative"}, {"aspect": "menu", "sentiment": "negative"}]}

Review: "The atmosphere was wonderful, but the service and food were terrible."

3. Prompt 3:

CoT:

Analyze the following review for aspects and sentiments step by step. The aspects can belong to the following categories: ambience, food, menu, miscellaneous, place, price, service, and staff. Follow these steps:

Step 1: Identify the aspects mentioned in the review.

- For example, aspects might include food, service, ambience, etc.

Step 2: Assign a sentiment (positive, negative, or neutral) to each aspect.

- For example, food might have a "positive" sentiment, service could be "negative", and ambience might be "neutral".

Step 3: Provide the output in JSON format.

Review: "The atmosphere was wonderful, but the service and food were terrible."

Dataset 2 was collected using web scraping. The first 300 comments from YouTube were scraped using the URL `video\_url = "https://www.youtube.com/watch?v=X9hKRIQ3uxc"` with the `webscrap.py` script, and the extracted data was saved in the `youtube\_comments\_english.json` file. Subsequently, the raw data was annotated using ChatGPT with separate prompts for zeroshot, fewshot, and CoT approaches.

1. Prompt 1:

Zero Shot:

Analyze the following review and identify the aspects discussed. The aspects can belong to the following categories: Recipe Ingredients, Recipe Instructions, Taste/Texture, Adjustments/Modifications, User Experience, Presentation/Visual Appeal, Questions/Clarifications, Sentiments Towards the Creator, Storage/Serving Suggestions, and Alternatives/Dietary Needs. For each aspect mentioned, assign a sentiment (positive, negative, or neutral). Provide the output in JSON format.

Example Review: "I used hot chocolate mix instead of cocoa powder because I don't have cocoa powder.. will it taste the same?"

2. Prompt 2:

Few Shot:

Analyze the following review for aspects and sentiments. The aspects can belong to the following categories: Recipe Ingredients, Recipe Instructions, Taste/Texture, Adjustments/Modifications, User Experience, Presentation/Visual Appeal, Questions/Clarifications, Sentiments Towards the Creator, Storage/Serving Suggestions, and Alternatives/Dietary Needs. For each aspect, determine its sentiment (positive, negative, or neutral). Refer to the examples for guidance. Provide the output in JSON format.

Example 1: Review: "I tried the recipe and it tasted amazing" Output: {"aspects": [{"category": "Taste/Texture", "sentiment": "positive"}]}

Example 2: Review: "This is the only baking channel I need." Output: {"aspects": [{"category": "Sentiments Towards the Creator", "sentiment": "positive"}]}

Example 3: Review: "I don't have any vanilla extract, would this affect the flavor of the brownies a lot?" Output: {"aspects": [{"category": "Questions/Clarifications", "sentiment": "neutral"}]}

Review: "I made this and it came out so well, it didn't even get some time to rest on the counter.

3. Prompt 3:

CoT:

Analyze the following review for aspects and sentiments step by step. The aspects can belong to the following categories: Recipe Ingredients, Recipe Instructions, Taste/Texture, Adjustments/Modifications, User Experience, Presentation/Visual Appeal, Questions/Clarifications, Sentiments Towards the Creator, Storage/Serving Suggestions, and Alternatives/Dietary Needs. Follow these steps:

Step 1: Identify the aspects mentioned in the review.

- For example, aspects might include Recipe Ingredients, Taste/Texture, User Experience, etc.

Step 2: Assign a sentiment (positive, negative, or neutral) to each aspect.

- For example, Recipe Ingredients might have a "positive" sentiment, Taste/Texture could be "negative", and User Experience might be "neutral".

Step 3: Provide the output in JSON format.

Review: "Will it taste great if I don't use the vanilla?"

These prompts were crafted to balance specificity and generality, allowing for diverse yet focused data collection.

3. Websites Used for Data Collection

The MAMS-ACSA Dataset and user reviews from a YouTube video served as the project's two primary data sources. Due to their depth and relevance to the project's goal of supporting Aspect-Based Sentiment Analysis (ABSA), both sources were chosen.

1. **MAMS-ACSA Dataset** (<https://github.com/siat-nlp/MAMS-for-ABSA/blob/master/data/MAMS-ACSA/raw/test.xml>)

One popular labeled dataset in the area of aspect-based sentiment analysis is the MAMS-ACSA dataset. This dataset was created especially to make it possible to analyze user evaluations according to particular criteria (contexts). This dataset was utilized as "Dataset 1" in the research, and its quality was assessed through annotations utilizing zero-shot, few-shot, and chain-of-thought prompting methodologies. This dataset was selected because to its trustworthiness and accessibility, and it is hosted on GitHub, a trustworthy open-access source.

2. **YouTube Comments** (<https://www.youtube.com/watch?v=X9hKRIO3uxc>)

For "Dataset 2," this YouTube video's user comments were utilized. This data source was chosen because it contains a variety of sentiments and natural user expressions. Web scraping techniques were used to gather comments on the video, which were then annotated with the same instructions. YouTube's API and site scraping tools were used to collect public comments in an ethical manner while following data collecting guidelines.

For the project's sentiment analysis and context-based evaluation, these two data sources offered substantial content. While YouTube comments provided a different perspective on the research by reflecting actual, real-world data, the MAMS-ACSA dataset was used as a standard dataset for evaluating model performance. When combined, they created a strong basis for evaluating the prompts' success and similarity rates.

4. Crawler Description

Instructions for Running the Crawler

xmltoJSON.py

This script converts an XML file into a JSON file.

Steps to Run

1. Prepare Input Data:

- Place the XML file (e.g., `test.xml`) in the `dataset/` directory.

2. Run the Script:

Navigate to the script's directory and execute:

```
python xmltoJSON.py
```

3. Output:

- The script reads the XML file and converts it into JSON format.
- The resulting JSON file (`test.json`) will be saved in the `dataset/` directory.

delete_annotated_data.py

This script extracts only the text fields from a JSON file and writes them as separate objects in a new JSON file.

Steps to Run

1. Prepare Input Data:

- Place the JSON file (`test.json`) in the `dataset/` directory.

2. Run the Script:

Execute the script:

```
python delete_annotated_data.py
```

3. Output:

- A new JSON file named `texts_separate.json` is created in the `dataset/` directory.
- This file contains each `text` field as a separate object.

first_100_texts.py

This script extracts the first 100 texts from a JSON file.

Steps to Run

1. Prepare Input Data:

- Place the JSON file (`texts_separate.json`) in the `dataset/` directory.

2. Run the Script:

- Execute the script:

```
python first_100_texts.py
```

3. Output:

- The script extracts the first 100 texts and saves them in `first_100_texts.json` in the `dataset/` directory.

first_100_fromtest.py

This script extracts the first 100 sentences from a JSON file containing sentences.

Steps to Run

1. Prepare Input Data:

- Place the input JSON file (`test.json`) in the `dataset/` directory.

2. Run the Script:

- Execute the script:

```
python first_100_fromtest.py
```

3. Output:

- The first 100 sentences are saved in `first_100_fromtest.json` in the `dataset/` directory.

task3.py

This script compares annotations between two datasets and calculates the success rate of annotation matching.

Steps to Run

1. Prepare Input Data:

- Place the required JSON files (`first_100_fromtest.json` and `full_annotated_data_cot.json`) in the `dataset/` directory.

2. Run the Script:

- Execute the script:

```
python task3.py
```

3. Output:

- The script calculates:
 - Success Rate: Percentage of correctly matched annotations.

- Total Annotations: Total number of annotations in the dataset.
- Correct Matches: Number of matches where the annotation matches the original data.

Example output:

Success Rate: 85.00%

Total Annotations: 200

Correct Matches: 170

task3_withSklearn.py

This script evaluates annotation performance using scikit-learn's `classification_report`.

Steps to Run

1. Prepare Input Data:

- Place the JSON files (`first_100_fromtest.json` and `full_annotated_data_cot.json`) in the `dataset/` directory.

2. Install Dependencies:

- Ensure `scikit-learn` is installed:
`pip install scikit-learn`

3. Run the Script:

- Execute the script:
`python task3_withSklearn.py`

4. Output:

- The script produces a classification report with metrics like precision, recall, and F1-score for sentiment classes (`positive`, `negative`, `neutral`, etc.).

Example output:

	precision	recall	f1-score	support
positive	0.88	0.85	0.86	120
negative	0.82	0.80	0.81	80
neutral	0.75	0.70	0.72	50

webscrap.py

The provided script `webscrap.py` is a YouTube comments crawler that scrapes and processes English comments from a specific video. Below are the steps to run the crawler:

Prerequisites

1. **Python Installation:** Ensure Python is installed on your system. Use version 3.7 or above.
 - Verify by running `python --version` in your terminal.
2. **Install Dependencies:** The script uses several libraries, including:
 - `selenium`: For web automation.
 - `langdetect`: For language detection.
 - Install these libraries using:

```
pip install selenium langdetect
```

Download WebDriver:

- The script uses **ChromeDriver** for Selenium. Download the appropriate version of ChromeDriver for your Chrome browser from [here](#).
- Place the driver in the specified path, e.g.,
`/Users/busedikici/Downloads/chromedriver-mac-arm64/chromedriver`.
- Update the path in the script if needed:
`driver_path = "path_to_your_chromedriver"`

Steps to Run

1. **Modify the YouTube Video URL:**
 - Update the `video_url` in the script with the URL of the video you want to scrape:
`video_url = "https://www.youtube.com/watch?v=your_video_id"`
 2. **Run the script:**
 - Navigate to the directory containing `webscrap.py`:
`cd path_to_directory`
 - Execute the script:
`python webscrap.py`
 3. **Output:**
 - The script scrolls through the YouTube comments section to load more comments dynamically.
 - It collects up to 300 English comments, processes them, and saves the data as a JSON file named `youtube_comments_english.json` in the same directory.
-

5. Success/Similarity Rate Analysis

```
PS C:\Users\gozde\OneDrive\Masaüstü\NEWNLP> & "C:/Program Files/Python312/python.exe" c:/Users/gozde/OneDrive/Masaüstü/NEWNLP/dataset/task3_withSklearn.py
```

	precision	recall	f1-score	support
positive	0.63	0.60	0.61	52
negative	0.84	0.62	0.72	79
neutral	0.25	0.03	0.05	99
None	0.00	0.00	0.00	0
micro avg	0.37	0.36	0.36	230
macro avg	0.43	0.31	0.35	230
weighted avg	0.54	0.36	0.41	230

Figure 1: Zero-shot prompting results with ChatGPT for Dataset 1 with SkLearn.

```
PS C:\Users\gozde\OneDrive\Masaüstü\NEWNLP> & "C:/Program Files/Python312/python.exe" c:/Users/gozde/OneDrive/Masaüstü/NEWNLP/dataset/task3_withSklearn.py
```

	precision	recall	f1-score	support
positive	0.63	0.60	0.61	52
negative	0.86	0.62	0.72	79
neutral	0.31	0.04	0.07	99
None	0.00	0.00	0.00	0
micro avg	0.37	0.37	0.37	230
macro avg	0.45	0.31	0.35	230
weighted avg	0.57	0.37	0.42	230

Figure 2: Few-shot prompting results with ChatGPT for Dataset 1 with SkLearn.

```
PS C:\Users\gozde\OneDrive\Masaüstü\NEWNLP> & "C:/Program Files/Python312/python.exe" c:/Users/gozde/OneDrive/Masaüstü/NEWNLP/dataset/task3_withSklearn.py
```

	precision	recall	f1-score	support
positive	0.64	0.62	0.63	52
negative	0.85	0.59	0.70	79
neutral	0.29	0.04	0.07	99
None	0.00	0.00	0.00	0
micro avg	0.37	0.36	0.36	230
macro avg	0.45	0.31	0.35	230
weighted avg	0.56	0.36	0.41	230

Figure 3: Chain-of-thought prompting results with ChatGPT for Dataset 1 with SkLearn.

```
PS C:\Users\gozde\OneDrive\Masaüstü\NEWNLP> & "C:/Program Files/Python312/python.exe" c:/Users/gozde/OneDrive/Masaüstü/NEWNLP/dataset/task3_withSklearn.py
```

	precision	recall	f1-score	support
positive	0.68	0.50	0.58	52
negative	0.67	0.28	0.39	79
neutral	1.00	0.02	0.04	99
None	0.00	0.00	0.00	0
accuracy			0.22	230
macro avg	0.59	0.20	0.25	230
weighted avg	0.81	0.22	0.28	230

Figure 4: Zero-shot prompting results with Gemini for Dataset 1 with SkLearn.

```
PS C:\Users\gozde\OneDrive\Masaüstü\NEWNLP> & "C:/Program Files/Python312/python.exe" c:/Users/gozde/OneDrive/Masaüstü/NEWNLP/dataset/task3_withSklearn.py
```

	precision	recall	f1-score	support
positive	0.68	0.50	0.58	52
negative	0.67	0.28	0.39	79
neutral	1.00	0.02	0.04	99
None	0.00	0.00	0.00	0
accuracy			0.22	230
macro avg	0.59	0.20	0.25	230
weighted avg	0.81	0.22	0.28	230

Figure 5: Few-shot prompting results with Gemini for Dataset 1 with SkLearn.

```
PS C:\Users\gozde\OneDrive\Masaüstü\NEWNLP> & "C:/Program Files/Python312/python.exe" c:/Users/gozde/OneDrive/Masaüstü/NEWNLP/dataset/task3_withSklearn.py
```

	precision	recall	f1-score	support
positive	0.68	0.50	0.58	52
negative	0.67	0.28	0.39	79
neutral	1.00	0.02	0.04	99
None	0.00	0.00	0.00	0
accuracy			0.22	230
macro avg	0.59	0.20	0.25	230
weighted avg	0.81	0.22	0.28	230

Figure 6: Chain-of-thought prompting results with Gemini for Dataset 1 with SkLearn.

```
/python.exe" c:/Users/gozde/OneDrive/Masaüstü/NEWNLP/dataset/task3_withSk
learn.py
```

	precision	recall	f1-score	support
positive	0.63	0.60	0.61	52
negative	0.80	0.62	0.70	79
neutral	0.40	0.04	0.07	99
None	0.00	0.00	0.00	0
micro avg	0.37	0.37	0.37	230
macro avg	0.46	0.31	0.35	230
weighted avg	0.59	0.37	0.41	230

Figure 7: Zero-shot prompting results with Claude for Dataset 1 with SkLearn.

```
PS C:\Users\gozde\OneDrive\Masaüstü\NEWNLP> & "C:/Program Files/Python312
/python.exe" c:/Users/gozde/OneDrive/Masaüstü/NEWNLP/dataset/task3_withSk
learn.py
```

	precision	recall	f1-score	support
positive	0.63	0.60	0.61	52
negative	0.80	0.62	0.70	79
neutral	0.38	0.03	0.06	99
None	0.00	0.00	0.00	0
micro avg	0.37	0.36	0.36	230
macro avg	0.45	0.31	0.34	230
weighted avg	0.58	0.36	0.40	230

Figure 8: Few-shot prompting results with Claude for Dataset 1 with SkLearn.

```
PS C:\Users\gozde\OneDrive\Masaüstü\NEWNLP> & "C:/Program Files/Python312
/python.exe" c:/Users/gozde/OneDrive/Masaüstü/NEWNLP/dataset/task3_withSk
learn.py
```

	precision	recall	f1-score	support
positive	0.63	0.60	0.61	52
negative	0.80	0.62	0.70	79
neutral	0.38	0.03	0.06	99
None	0.00	0.00	0.00	0
micro avg	0.37	0.36	0.36	230
macro avg	0.45	0.31	0.34	230
weighted avg	0.58	0.36	0.40	230

Figure 9: Chain-of-thought prompting results with Claude for Dataset 1 with SkLearn.

6. Conclusion

In this project, we explored and compared different prompting methods and language models for Aspect-Based Sentiment Analysis (ABSA), focusing on ChatGPT, Gemini, and Claude. Our results showed that there were clear differences in performance, and each model had its strengths and weaknesses.

ChatGPT stood out as the best-performing model, especially when we used the few-shot prompting method. By including examples in the prompt, we were able to achieve higher precision, recall, and F1-scores for positive and negative sentiments. Few-shot prompting proved to be the most effective strategy overall.

Gemini, on the other hand, performed the worst, particularly with chain-of-thought prompting. Although it showed high precision for neutral sentiments in some cases, its recall was extremely low, which resulted in very poor F1 scores and inconsistent results. Chain-of-thought prompting, in general, didn't work as well as we had hoped, across all models.

Claude delivered decent results and had better-weighted averages than Gemini, but it still couldn't match ChatGPT. It struggled with annotating neutral sentiments and had lower recall for more complex or niche-specific cases, even though its zero-shot and few-shot results were relatively balanced.

One of the key components of this study was leveraging Dataset 2, a real-world dataset collected via YouTube. This dataset introduced user-generated content with diverse and unstructured sentiment expressions. Incorporating Dataset 2 allowed us to evaluate the adaptability of the prompting methods in handling natural language variations from an unstructured environment. While ChatGPT performed well on this dataset, the same challenges persisted, particularly in annotating neutral sentiments accurately. Despite this, the integration of Dataset 2 strengthened the robustness of our evaluation and provided practical insights into how these models perform with real-world data.

In conclusion, ChatGPT with few-shot prompting gave the best results in our experiments, balancing accuracy and consistency. For future projects, it would be helpful to focus on improving how neutral sentiments are handled, creating more specific prompts for certain domains, and testing the methods on other datasets to see if the results hold up. The addition of Dataset 2 demonstrated the value of including real-world, user-generated content for testing and provided deeper insights into the strengths and limitations of modern NLP tools in ABSA tasks.

7. References

- Pfalzgraf, B. (2020, April 29). *How to Use Selenium to Web-Scrape with Example*. Medium. <https://towardsdatascience.com/how-to-use-selenium-to-web-scrape-with-example-80f9b23a843a>
- Zdrok, O. (2024, March 19). *7 Contrasts Between Zero-Shot and Few-Shot Prompting*. Shelf. <https://shelf.io/blog/zero-shot-and-few-shot-prompting/>